

Асинхронне програмування в JavaScript

1. Колбеки (Callbacks)

Базове використання

```
function fetchData(callback) {  
  setTimeout(() => {  
    callback('Дані отримано');  
  }, 1000);  
}  
  
fetchData((data) => {  
  console.log(data);  
});
```

Обробка помилок

```
function fetchData(success, error) {  
  const random = Math.random();  
  setTimeout(() => {  
    if (random > 0.5) {  
      success('Дані отримано');  
    } else {  
      error('Помилка отримання даних');  
    }  
  }, 1000);  
}
```

2. Проміси (Promises)

Створення промісів

```
const promise = new Promise((resolve, reject) => {  
  const random = Math.random();  
  setTimeout(() => {  
    if (random > 0.5) {  
      resolve('Успіх!');  
    } else {  
      reject('Помилка!');  
    }  
  }, 1000);  
});
```

Ланцюжки промісів

```
fetchUser(1)
  .then(user => fetchUserPosts(user.id))
  .then(posts => fetchPostComments(posts[0].id))
  .catch(error => console.error(error));
```

Методи Promise

Promise.all

Виконує масив промісів паралельно і повертає масив результатів. Якщо хоча б один проміс завершиться з помилкою, весь **Promise.all** завершиться з цією помилкою.

```
const promises = [
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
  fetch('/api/comments').then(r => r.json())
];

Promise.all(promises)
  .then([users, posts, comments]) => {
    console.log('Всі дані:', { users, posts, comments });
  })
  .catch(error => {
    console.error('Помилка в одному з промісів:', error);
  });
```

Promise.race

Повертає результат першого завершеного промісу (незалежно від того, успішно він виконався чи з помилкою).

```
const promises = [
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
  fetch('/api/comments').then(r => r.json())
];

Promise.race(promises)
  .then(data => {
    console.log('Перший завершений проміс:', data);
  })
  .catch(error => {
    console.error('Перший завершений проміс з помилкою:', error);
  });
```

Promise.allSettled

Чекає завершення всіх промісів і повертає масив об'єктів з результатами (як успішними, так і помилковими).

```
const promises = [
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
  fetch('/api/comments').then(r => r.json())
];

Promise.allSettled(promises)
  .then(results => {
    results.forEach((result, index) => {
      if (result.status === 'fulfilled') {
        console.log(`Проміс ${index} успішно завершено`, result.value);
      } else {
        console.error(`Проміс ${index} завершено з помилкою`, result.reason);
      }
    });
  });
```

Promise.any

Повертає результат першого успішно завершеного промісу. Якщо всі проміси завершаться з помилкою, повернеться `AggregateError`.

```
const promises = [
  fetch('/api/users').then(r => r.json()),
  fetch('/api/posts').then(r => r.json()),
  fetch('/api/comments').then(r => r.json())
];

Promise.any(promises)
  .then(data => {
    console.log('Перший успішний проміс:', data);
  })
  .catch(error => {
    console.error('Всі проміси завершилися з помилкою:', error);
  });
```

3. Async/Await

Основи async/await

```
async function fetchUserData() {  
  try {  
    const response = await fetch('https://api.example.com/user');  
    const data = await response.json();  
    return data;  
  } catch (error) {  
    console.error('Помилка:', error);  
  }  
}
```

Паралельне виконання

```
async function fetchAllData() {  
  const [users, posts, comments] = await Promise.all([  
    fetch('/api/users').then(r => r.json()),  
    fetch('/api/posts').then(r => r.json()),  
    fetch('/api/comments').then(r => r.json())  
  ]);  
  
  return { users, posts, comments };  
}
```